

# VN Party — Testing Guide

Author: Lê Quan Phát Thành (ITITI21312)

Institution: International University — VNU HCMC

This guide is for supervisors and committee members who wish to run the system, verify the game, and reproduce the three hypothesis experiments (H1, H2, H3).

## Part 1 — Playing the Game (Deployed Version)

The system is already deployed on the internet. No installation is required.

| App                                                                                                | URL                                                                                               |
|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|  Host / TV Screen | <a href="https://vn-party-host.vercel.app">https://vn-party-host.vercel.app</a>                   |
|  Player (Mobile)  | <a href="https://vn-party-player.vercel.app">https://vn-party-player.vercel.app</a>               |
|  Backend API      | <a href="https://vn-party-thesis-9lo5.onrender.com">https://vn-party-thesis-9lo5.onrender.com</a> |

### How to Start a Game

1. Open the Host screen ([vn-party-host.vercel.app](https://vn-party-host.vercel.app)) on a laptop or TV browser.
2. Open the Player app ([vn-party-player.vercel.app](https://vn-party-player.vercel.app)) on a phone (or a second browser tab).
3. On the Player screen, enter a nickname and enter the 6-character Room Code shown on the Host screen.
4. Once all players have joined, the Host player presses “Start Game” from the player app (the person with the  Host badge).
5. Each round: players read the question on the Host screen and submit their answer on their phone within the time limit.

### Game Rules Summary

- 4–8 players per room.
- Each correct guess earns points. Wrong guesses earn “charges” used for distortion powers.
- Distortion Powers (used during the Results phase between rounds):

| Power                 | Cost      | Limit                         |
|-----------------------|-----------|-------------------------------|
| Remove Option         | 2 charges | Max 1 use/player, 3 uses/room |
| Swap Category         | 2 charges | Max 2/player, 4/room          |
| Force Blind (shuffle) | 3 charges | Max 1/player, 3/room          |
| Inject Fake Option    | 4 charges | Max 1/player                  |
| Merge Realities       | 4 charges | Max 1 per entire game         |

 **Note on Remove Option:** If 3 different players all use “Remove Option” on the same target player in the same round, it removes all 3 incorrect answers — leaving only the correct one visible. This is a documented design paradox discussed in the thesis (Section 6.2).

## Part 2 — If the Deployed Links Don't Work

Render's free tier has limited monthly usage. If the backend is down (connection error), you can run everything locally.

### Prerequisites

- Elixir 1.16+ — <https://elixir-lang.org/install.html>
- Node.js 20+ — <https://nodejs.org/>
- Docker Desktop — <https://www.docker.com/products/docker-desktop/>

### Step 1 — Start the Databases

```
cd vn-party-thesis
docker-compose up -d
```

### Step 2 — Start the Backend

```
cd backend
mix deps.get
mix ecto.create
mix ecto.migrate
mix phx.server
```

Backend starts at <http://localhost:4000>

### Step 3 — Start the Player Frontend

```
cd frontend-player
npm install
npm run dev
```

Opens at <http://localhost:5173>

### Step 4 — Start the Host Frontend

```
cd frontend-host
npm install
npm run dev
```

Opens at <http://localhost:5174>

### Local URLs

| App                                                                                             | URL                                                       |
|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
|  Host Screen | <a href="http://localhost:5174">http://localhost:5174</a> |
|  Player      | <a href="http://localhost:5173">http://localhost:5173</a> |

## Part 3 — Reproducing the Hypothesis Experiments

### H1 — Latency Test

Hypothesis: p95 latency for player answer submission stays  $\leq 300\text{ms}$  under real-world conditions.

## Step 1 — Install the Node.js load generator

```
cd tools/loadgen
npm install
```

## Step 2 — Run the baseline test

```
node tools/loadgen/latency_loadgen.mjs \
  --api https://vn-party-thesis-9lo5.onrender.com/api \
  --ws wss://vn-party-thesis-9lo5.onrender.com/socket/websocket \
  --players 8 \
  --messages 100 \
  --interval-ms 500 \
  --mode truth_collapse
```

The script prints: room=XXXXXX — note the room code.

## Step 3 — Export the latency data

```
cd backend
mix telemetry.export_latency XXXXXX \
  --event submit_prediction \
  --out ../analysis/out_baseline.csv
```

## Step 4 — Analyze results and generate chart

```
source analysis/.venv/bin/activate
python3 analysis/latency_analyze.py \
  --in analysis/out_baseline.csv \
  --outdir analysis/results \
  --scenario baseline \
  --event submit_prediction
```

Results saved to analysis/results/.

**Pass criteria:  $p95 \leq 300\text{ms}$**

## H2 — Scalability Test

Hypothesis: The system supports  $\geq 500$  concurrent game rooms with  $p95$  latency  $\leq 300\text{ms}$  and error rate  $< 1\%$ .

⚠️ *This test runs for ~37 minutes and requires k6 to be installed — <https://k6.io/docs/getting-started/installation/>*

## Run the full H2 load test

```
k6 run \
  --out json=analysis/k6_h2_final_out.json \
  -e API_BASE=https://vn-party-thesis-9lo5.onrender.com/api \
  -e WS_BASE=wss://vn-party-thesis-9lo5.onrender.com/socket/websocket \
  loadtest/k6/h2_load_test.js
```

## After the test finishes, generate the chart

```
source analysis/.venv/bin/activate
python3 analysis/h2_k6_analyze.py \
  --in analysis/k6_h2_final_out.json \
  --outdir analysis/h2_results
```

### Results location:

- analysis/h2\_results/h2\_summary.txt — p50/p95/p99 latency + error rate
- analysis/h2\_results/h2\_latency\_timeseries.png — latency chart over time

**Pass criteria: Both thresholds marked ✓ in the k6 terminal output.**

## H3 — Fairness / Anti-Cheat Test

Hypothesis: The commit-reveal protocol detects  $\geq 95\%$  of cheating attempts across 4 attack types.

### Step 1 — Start local database

```
docker-compose up -d
```

### Step 2 — Run the attack simulation (400 attempts across 4 attack types)

```
cd backend
MIX_ENV=test mix ecto.create
MIX_ENV=test mix ecto.migrate
MIX_ENV=test mix test test/h3_attack_sim_test.exs --trace
```

### Attack scenarios tested

| Scenario            | What it simulates                            | Expected detection        |
|---------------------|----------------------------------------------|---------------------------|
| hash_tampering      | Attacker modifies stored answer after commit | is_valid = false, flagged |
| replay_attack       | Reuse the same hash in a new round           | {:error, :replay_attack}  |
| late_commit         | Submit after the round timer closes          | {:error, :late_commit}    |
| timing_manipulation | Commit in the last milliseconds              | Flagged suspicious        |

### Expected output

```
=== H3 Attack Simulation Results ===
hash_tampering:      detected=100/100 rate=100.0%
replay_attack:       detected=100/100 rate=100.0%
late_commit:         detected=100/100 rate=100.0%
timing_manipulation: detected=100/100 rate=100.0%
overall:             detected=400/400 rate=100.0%
```

### Bonus — Verify the Blockchain Audit Trail

```
curl -s "https://vn-party-thesis-9lo5.onrender.com/api/rooms/ROOMCODE/audit" |
python3 -m json.tool
```

This returns the SHA-256 hash chain for all game events. Any modification to a past event breaks the chain and is mathematically detectable.

## Part 4 — Blockchain Performance Benchmark

```
cd backend
MIX_ENV=test mix test test/blockchain_performance_test.exs --trace
```

Prints a formatted table comparing:

- ETS in-memory audit trail (simulated mode) vs.
- Direct PostgreSQL write speed
- Concurrent room throughput simulation

## Quick Reference

| What                         | Command / URL                                               |
|------------------------------|-------------------------------------------------------------|
| Play game (deployed)         | vn-party-host.vercel.app + vn-party-player.vercel.app       |
| API health check             | curl https://vn-party-thesis-9lo5.onrender.com/api/health   |
| H1 Latency test              | node tools/loadgen/latency_loadgen.mjs --api ... --ws ...   |
| H2 Scalability test (37 min) | k6 run loadtest/k6/h2_load_test.js                          |
| H3 Security test             | cd backend && mix test test/h3_attack_sim_test.exs          |
| Blockchain benchmark         | cd backend && mix test test/blockchain_performance_test.exs |
| Audit trail for a game       | GET /api/rooms/:code/audit                                  |

# Hướng Dẫn Kiểm Tra — Phiên Bản Tiếng Việt

Tác giả: Lê Quan Phát Thành (ITITIU21312)  
Trường: Đại học Quốc tế — ĐHQG TP.HCM

Tài liệu này dành cho giảng viên hướng dẫn và hội đồng phản biện muốn chạy hệ thống, xác minh trò chơi và tái hiện ba thí nghiệm giả thuyết (H1, H2, H3).

## Phần 1 — Chơi Thử (Phiên Bản Đã Deployed)

Hệ thống đã được Deployed lên internet. Không cần cài đặt thêm bất cứ thứ gì.

| App                                                                                                | URL                                       |
|----------------------------------------------------------------------------------------------------|-------------------------------------------|
|  Host / TV Screen | https://vn-party-host.vercel.app          |
|  Player (Mobile)  | https://vn-party-player.vercel.app        |
|  Backend API      | https://vn-party-thesis-9lo5.onrender.com |

### Cách Bắt Đầu Một Ván Chơi

- Mở màn hình Host (vn-party-host.vercel.app) trên máy tính hoặc TV.
- Mở ứng dụng Người chơi (vn-party-player.vercel.app) trên điện thoại (hoặc tab trình duyệt thứ hai).
- Trên màn hình Người chơi: nhập biệt danh và nhập mã phòng 6 ký tự hiển thị trên màn hình Host.
- Sau khi tất cả người chơi đã vào, người chơi có huy hiệu 🗝️ Host nhấn “Bắt đầu trò chơi” từ ứng dụng điện thoại.
- Mỗi vòng: người chơi đọc câu hỏi trên màn hình Host và gửi câu trả lời trong thời gian quy định.

### Tóm Tắt Luật Chơi

- 4–8 người chơi mỗi phòng.
- Trả lời đúng sẽ được điểm. Trả lời sai sẽ nhận được “charges” để dùng các hiệu ứng bóp méo.
- Hiệu ứng bóp méo (Truth Distortion) — sử dụng trong giai đoạn Kết quả giữa các vòng:

| Hiệu ứng                            | Chi phí   | Giới hạn                             |
|-------------------------------------|-----------|--------------------------------------|
| Xóa đáp án (Remove Option)          | 2 charges | Tối đa 1 lần/người chơi, 3 lần/phòng |
| Đổi chủ đề (Swap Category)          | 2 charges | 2/người, 4/phòng                     |
| Mù quáng (Force Blind)              | 3 charges | 1/người, 3/phòng                     |
| Chèn đáp án giả (Inject Fake)       | 4 charges | 1/người                              |
| Hợp nhất thực tại (Merge Realities) | 4 charges | Tối đa 1 lần cả ván                  |

⚠ Lưu ý về Xóa đáp án: Nếu 3 người chơi khác nhau cùng dùng “Xóa đáp án” nhắm vào một người trong cùng một vòng, cả 3 đáp án sai sẽ bị xóa — chỉ còn lại đáp án đúng. Đây là một nghịch lý thiết kế được ghi lại trong luận văn (Mục 6.2).

## Phần 2 — Nếu Đường Dẫn Deployed Không Hoạt Động

Render sử dụng gói miễn phí có giới hạn hàng tháng. Nếu backend bị ngừng, bạn có thể chạy toàn bộ hệ thống cục bộ trên máy tính của mình.

### Yêu Cầu Cài Đặt

- Elixir 1.16+ — <https://elixir-lang.org/install.html>
- Node.js 20+ — <https://nodejs.org/>
- Docker Desktop — <https://www.docker.com/products/docker-desktop/>

### Bước 1 — Khởi động Cơ Sở Dữ Liệu

```
cd vn-party-thesis
docker-compose up -d
```

### Bước 2 — Khởi động Backend

```
cd backend
mix deps.get
mix ecto.create
mix ecto.migrate
mix phx.server
```

Backend chạy tại <http://localhost:4000>

### Bước 3 — Khởi động Frontend Người Chơi

```
cd frontend-player
npm install
npm run dev
```

Mở tại <http://localhost:5173>

### Bước 4 — Khởi động Frontend Host

```
cd frontend-host
npm install
npm run dev
```

Mở tại <http://localhost:5174>

### URL Cục Bộ

| App                                                                                             | URL                                                       |
|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
|  Host Screen | <a href="http://localhost:5174">http://localhost:5174</a> |
|  Player      | <a href="http://localhost:5173">http://localhost:5173</a> |

## Phần 3 — Tái Hiện Các Thí Nghiệm Giả Thuyết

### H1 — Kiểm Tra Độ Trễ (Latency)

Giả thuyết: Độ trễ p95 của việc gửi câu trả lời người chơi duy trì  $\leq 300\text{ms}$  trong điều kiện thực tế.

#### Bước 1 — Cài đặt công cụ tạo tải (Node.js)

```
cd tools/loadgen
npm install
```

#### Bước 2 — Chạy thử nghiệm cơ sở

```
node tools/loadgen/latency_loadgen.mjs \
  --api https://vn-party-thesis-9lo5.onrender.com/api \
  --ws wss://vn-party-thesis-9lo5.onrender.com/socket/websocket \
  --players 8 \
  --messages 100 \
  --interval-ms 500 \
  --mode truth_collapse
```

Script sẽ in ra: room=XXXXXX — ghi lại mã phòng này.

#### Bước 3 — Xuất dữ liệu độ trễ từ backend

```
cd backend
mix telemetry.export_latency XXXXXX \
  --event submit_prediction \
  --out ../analysis/out_baseline.csv
```

#### Bước 4 — Phân tích kết quả và tạo biểu đồ

```
source analysis/.venv/bin/activate
python3 analysis/latency_analyze.py \
  --in analysis/out_baseline.csv \
  --outdir analysis/results \
  --scenario baseline \
  --event submit_prediction
```

Kết quả lưu tại analysis/results/.

**Tiêu chí đạt:  $p95 \leq 300\text{ms}$**

### H2 — Kiểm Tra Khả Năng Mở Rộng (Scalability)

Giả thuyết: Hệ thống hỗ trợ  $\geq 500$  phòng chơi đồng thời với  $p95 \leq 300\text{ms}$  và tỷ lệ lỗi  $< 1\%$ .

⚠ Bài kiểm tra này chạy trong khoảng ~37 phút và yêu cầu cài đặt k6 — <https://k6.io/docs/getting-started/installation/>

#### Chạy bài kiểm tra tải H2 đầy đủ

```
k6 run \
  --out json=analysis/k6_h2_final_out.json \
  -e API_BASE=https://vn-party-thesis-9lo5.onrender.com/api \
  -e WS_BASE=wss://vn-party-thesis-9lo5.onrender.com/socket/websocket \
  loadtest/k6/h2_load_test.js
```

#### Sau khi bài kiểm tra hoàn thành, tạo biểu đồ

```
source analysis/.venv/bin/activate
python3 analysis/h2_k6_analyze.py \
```



```
--in analysis/k6_h2_final_out.json \  
--outdir analysis/h2_results
```

#### Vị trí kết quả:

- analysis/h2\_results/h2\_summary.txt — độ trễ p50/p95/p99 và tỷ lệ lỗi
- analysis/h2\_results/h2\_latency\_timeseries.png — biểu đồ độ trễ theo thời gian

**Tiêu chí đạt:** Cả hai ngưỡng được đánh dấu ✓ trong kết quả k6.

## H3 — Kiểm Tra Tính Công Bằng / Chống Gian Lận

Giả thuyết: Giao thức commit-reveal phát hiện  $\geq 95\%$  các hành vi gian lận trên 4 loại tấn công.

### Bước 1 — Khởi động cơ sở dữ liệu cục bộ

```
docker-compose up -d
```

### Bước 2 — Chạy mô phỏng tấn công (400 lần thử trên 4 loại)

```
cd backend  
MIX_ENV=test mix ecto.create  
MIX_ENV=test mix ecto.migrate  
MIX_ENV=test mix test test/h3_attack_sim_test.exs --trace
```

#### Các kịch bản tấn công được kiểm tra

| Kịch bản            | Mô phỏng                                      | Kết quả mong đợi         |
|---------------------|-----------------------------------------------|--------------------------|
| hash_tampering      | Kẻ tấn công sửa câu trả lời sau khi đã commit | Bị gán is_valid = false  |
| replay_attack       | Dùng lại cùng hash commit ở vòng khác         | {:error, :replay_attack} |
| late_commit         | Gửi sau khi bộ đếm vòng đã đóng               | {:error, :late_commit}   |
| timing_manipulation | Gửi trong những mili-giây cuối cùng           | Bị gán chờ ngắt ngữ      |

#### Kết quả mong đợi trên terminal

```
=== H3 Attack Simulation Results ===  
hash_tampering:      detected=100/100 rate=100.0%  
replay_attack:       detected=100/100 rate=100.0%  
late_commit:         detected=100/100 rate=100.0%  
timing_manipulation: detected=100/100 rate=100.0%  
overall:             detected=400/400 rate=100.0%
```

### Thêm — Xác minh Blockchain Audit Trail

```
curl -s "https://vn-party-thesis-9lo5.onrender.com/api/rooms/ROOMCODE/audit" |  
python3 -m json.tool
```

Trả về chuỗi hash SHA-256 cho toàn bộ sự kiện trong ván chơi. Bất kỳ sửa đổi nào đối với sự kiện trong quá khứ đều phá vỡ chuỗi và có thể phát hiện bằng toán học.

## Phần 4 — Benchmark Hiệu Năng Blockchain

```
cd backend  
MIX_ENV=test mix test test/blockchain_performance_test.exs --trace
```

In ra bảng benchmark định dạng so sánh:

- Tốc độ audit trail trong bộ nhớ ETS (chế độ mô phỏng) so với
- Tốc độ ghi trực tiếp vào PostgreSQL
- Thông lượng mô phỏng phòng đồng thời

## Tham Khảo Nhanh

| Mục đích                 | Lệnh / Đường dẫn                                            |
|--------------------------|-------------------------------------------------------------|
| Chơi game (đã Deployed)  | vn-party-host.vercel.app + vn-party-player.vercel.app       |
| Kiểm tra API hoạt động   | curl https://vn-party-thesis-9lo5.onrender.com/api/health   |
| Kiểm tra H1 (Độ trễ)     | node tools/loadgen/latency_loadgen.mjs --api ... --ws ...   |
| Kiểm tra H2 (37 phút)    | k6 run loadtest/k6/h2_load_test.js                          |
| Kiểm tra H3 (Bảo mật)    | cd backend && mix test test/h3_attack_sim_test.exs          |
| Benchmark Blockchain     | cd backend && mix test test/blockchain_performance_test.exs |
| Audit trail một ván chơi | GET /api/rooms/:code/audit                                  |